

Arthax Android App — How It Works

Simple guide to the full flow: finding calls, checking if the number is a lead, finding the recording, and uploading to the CRM. Written from the current code (branch `master`, including uncommitted changes).

1. The big picture (one sentence each)

1. **Rep signs in** with phone number + OTP (code arrives on registered email). The app saves the token securely and marks "start tracking from the newest call already on the phone".
2. **A background watcher stays running** (foreground service with a notification) and watches the phone's **system call log**.
3. **When any call ends** (incoming or outgoing, from our app or the normal dialler), the app reads the new call log rows.
4. For each new call it **asks the CRM live: "is this number a lead?"**
5. **Not a lead** → ignored completely (no number saved, no recording touched, nothing uploaded).
6. **Lead** → if the call connected, find its **recording file** in the folder the rep chose, copy it into the app, and put the call in an **upload queue**.
7. **Upload in 2 steps**: create the call record in the CRM, then attach the recording. Failures are retried automatically.



Main files: `CallMonitorService` (watcher) · `CallLogReconciler` (the brain) · `CallLogReader` · `LeadResolver` · `RecordingHarvester` + `CallWindow` · `CallSyncRepository` + `PendingCallStore` · `WorkScheduler` + workers.

2. Setup & picking leads to call

- **Login**: `send-otp` → `login`. Phone is cleaned to the last 10 digits. On success: token saved in encrypted storage, call-tracking "watermark" armed, watcher service started, 15-minute safety job scheduled.
- **Onboarding** needs: call permissions (incl. `READ_CALL_LOG`) and the **recordings folder** (rep picks it once via the system folder picker; the app has access to only that folder). Battery optimisation / autostart are advised but not blocking.
- **Leads screen**: loads the rep's leads from `GET /api/leads/`, 100 per page, `is_junk=false`. No local lead database — always fresh from the server. Search goes to the server (350 ms debounce). Leads without a phone number are hidden. List auto-refreshes when you come back to the screen if it is older than 15 s.
- **Call button**: "Direct" mode dials immediately, "Dialler" mode opens the phone dialler. **Important**: tapping Call does not decide which lead the call belongs to. Every call is matched later from the call log by number — the Call button is just a shortcut.

3. How we detect that a call happened

Android does not tell apps "the call was answered", and the phone-state broadcast is unreliable on Xiaomi/Oppo. So the **system call log is the single source of truth**: it has the number, direction (type), start time, and real talk duration (0 = not answered).

Trigger (what wakes up a check)	When
Call-log change observer	Waits 1.5 s for the row to settle (max 5 s if the log keeps changing), then checks.
Line goes idle (hang-up listener)	Extra quick checks at 2 s, 6 s, 15 s, 40 s after hang-up — for phones (e.g. Samsung) that write the log late.
Heartbeat inside the service	Every 1 minute .
App opened / resumed, phone reboot, sign-in, <code>PHONE_STATE</code> broadcast	Restart the service → one catch-up check.
WorkManager safety job	Every 15 min; plus a "run when back online" job after offline failures.

Only **one check runs at a time** (`ReconcileGate` + a mutex). If a call ends while a check is running, the trigger is remembered and the check runs again right after — so no trigger is lost.

4. The reconcile algorithm (the brain) — CallLogReconciler

4.1 Which call-log rows are read

The app keeps three saved markers:

- **Time watermark** — start time of the newest call already handled.
- **ID watermark** — highest call-log row id already handled.
- **Floor** — set once at first run; nothing older is ever posted (stops old history being uploaded).

Query: rows where `id > idWatermark` **OR** `date > max(floor, timeWatermark - 6 hours)`, oldest first, max 500 rows.

Why both? ID catches overlapping calls (a row written later but with an earlier start time). Date catches Xiaomi-style phones that *re-date an existing row* for a repeat missed call instead of adding a new one. The **6-hour look-back** re-reads recent history every pass so a late or out-of-order row can never be skipped forever.

On first sign-in the watermark is set to the **newest call already on the phone** (not "now"), so the first call after login is not missed and old calls are not uploaded. Sign-out clears all markers.

4.2 For each row, in order

1. **Already handled?** A call's identity = **row id + start time**. Skip if it is in the upload queue (`PendingCallStore`) or in the "checked, not a lead" list (`SeenCallStore`, last 2,000 rows).
2. **Ask if it's a lead** (section 5). Same number twice in one pass = asked only once.
3. **Result:**
 - **Not a lead** → add row (id+time only, no number) to the seen list. Nothing else.
 - **Unavailable** (offline / server error) → **keep it, never drop it**. Watermarks stop moving at this row, and a "run when online" job is scheduled. The row will be checked again.
 - **Lead** → find recording (only if connected) → create a `PendingCall` in the queue → send it immediately.
4. Move watermarks forward only past rows that are settled.

4.3 What is saved for a lead call

Field	Rule
connected	<code>duration > 0</code> AND type is not MISSED and not REJECTED
direction	OUTGOING type → <code>outbound</code> , everything else → <code>inbound</code>
duration / times	Real talk seconds from the log; end = start + duration
external_id	<code>android-<uuid></code> , created once and reused on every retry → server can de-duplicate
owner_user_id	The signed-in rep, so another rep on the same phone never uploads it under their token

5. How we decide "is this number a lead?" — LeadResolver

1. **Make a match key:** keep only digits, take the **last 10**. (+91 98765-43210, 098765 43210, 9876543210 → all 9876543210.) Empty/hidden number → **Not a lead**.
2. **Ask the CRM live, every time:** GET `/api/leads/?search=<key>&limit=25` with `is_junk` not sent (so a call to a lead later marked junk still counts).
3. **Don't trust the search order.** The search is fuzzy text (notes etc. can match). Keep only results whose own `phone` gives the **same 10-digit key**.
 - 0 matches → **Not a lead** (and remove it from offline cache).
 - 1 match → that lead.
 - 2+ matches (duplicate leads) → pick in this order: **assigned to me** → **not junk** → **newest created**. A warning is logged.
4. **Match found** → save number→lead in a small offline cache (max 1,000, valid 7 days).
5. **CRM unreachable** → if the offline cache says it was a lead in the last 7 days, treat as **Lead** (so the recording is captured before the phone deletes it). Otherwise → **Unavailable** (retry later). A non-lead is never cached, so a newly added lead is found on the very next call.

6. Recording algorithm — `RecordingHarvester` + `CallWindow`

Only for **connected lead calls** that ended in the **last 6 hours**. Needs the recordings folder to be selected and still accessible (otherwise the call is still uploaded, just without audio, and an error is logged).

1. **Scan the folder** (plus up to 2 sub-folder levels, max 200 files) for audio files (by MIME type or extension: m4a, mp3, amr, aac, wav, ogg, opus, 3gp, awb...).
2. **Time window filter** — file's "last modified" must be between:
 - **From:** call start – 30 s
 - **To:** call end + 90 s, but **not later than next call start + 10 s** (so a call can't steal the next call's audio).
3. **Rank the files (lowest score wins, not "newest"):**
 - a. File name contains **this phone number** (a digit run of 10–15 digits ending with the key) → best.
 - b. No number in name → next best.
 - c. File modified after the next call had started → worst (probably belongs to the next call).

Inside each group: closest to the call's end time wins.

4. **Skip** files already attached to another call (same source URI).
5. **Wait until the file is finished:** poll every 2 s; accept only when the **size is the same on two polls in a row**. Give up after **60 s** (setting, 15–180 s).
6. **Copy into app private storage** (write `.part` then rename). This copy survives the phone's recorder deleting the original. A warning is logged if the file looks too small for the call length.

7. Upload & retry algorithm — `CallSyncRepository`

States of a queued call



(any step can go to **FAILED**)

1. POST `/api/calls/` with `lead_id`, `call_status` = `connected` / `not_answered`, `outcome` = `connected` / `not_picked`, `direction`, `duration`, `created_at` (= end time), `external_id`. Save the returned server call id.
2. If there is a recording: POST `/api/calls/{id}/upload-recording` (multipart field `file`). On success → DONE and **delete the local copy**.

Each step is saved to disk before the next, so a crash resumes from the same step without double-posting.

Sending & retry

- Right after detection the call is **sent immediately** from the service (0.5 s gap between calls in a batch).
- If that doesn't finish → a `WorkManager` job per call: needs internet, exponential backoff starting 10 s.
- **Retry** on: no network, timeout, 5xx, 429/408/425. Max **8 attempts**, then FAILED + notification.
- **Fail immediately** on: 4xx rejection, 401/403, 5xx that says `ffmpeg` / `invalid data`, or local file missing.
- Stuck calls are re-queued on every app start and every reconcile pass (only for the signed-in rep).
- DONE rows are kept 3 days (for duplicate protection), then removed. Rep can retry/discard FAILED calls from the Activity screen.

8. Things we noticed while verifying the code

- **Recording search blocks the next check.** The up-to-60 s recording wait runs inside the reconcile lock. Back-to-back connected calls are not lost (the gate re-runs), but a second call may reach the CRM up to about a minute later.
- **Expired session = permanent failure.** A 401 marks the call FAILED (not retried) and signs the rep out. After signing in again, the rep has to press retry on the Activity screen.
- **Missed vs rejected is not sent.** Every unanswered call goes as `not_answered`; the `missed` / `rejected` statuses exist in code but are never used.
- **Size limits.** Queue file holds max 200 calls (oldest dropped, even if still unsent, e.g. after days offline). The call-log read is capped at 500 rows in the 6 h window — only a problem for very high call volume.
- **Calls while signed out are never uploaded** — sign-in re-arms tracking from the newest existing call. (By design.)
- Small stale comments: `CallSyncRepository` mentions `CallCompleter.decideOutcome` (no longer exists); `LeadsViewModel.onCallLaunchFailed` says "undo the session" (there is no session any more).

Privacy summary: non-lead calls leave no number, name or audio on the device or server — only an anonymous row id + time so they aren't checked twice. The app can read only the one folder the rep picked.